

Knowledge Distillation for Edge-Friendly Plant Disease Classification

Progress Checkpoint Report
Machine Learning Term Project

Meric Kelemeden
Department of Computer Engineering

May 9, 2026

Contents

1	Abstract	3
2	Introduction	3
3	Related Work	3
3.1	Plant Disease Classification	3
3.2	Knowledge Distillation	3
3.3	Efficient Architectures for Edge Deployment	4
4	Dataset and Preprocessing	4
4.1	PlantVillage	4
4.2	Train / Validation / Test Split	4
4.3	Preprocessing and Augmentation	5
5	Methodology	5
5.1	Model Architecture	5
5.2	Supervised Baseline Training	5
5.3	Distillation Loss (Next Phase)	5
6	Experiments and Results	6
6.1	Experiment Configuration	6
6.2	Training Dynamics	6
6.3	Test Set Evaluation	6
6.4	Summary Metrics	7
6.5	Confusion Matrix	7
6.6	Planned Next-Phase Comparison	8
7	Implementation Details	8
7.1	Repository Structure	8
7.2	Reproducibility	8

1 Abstract

Deep convolutional neural networks achieve strong accuracy on image classification tasks but are often too resource-intensive for edge deployment. This project investigates knowledge distillation as a model-compression technique for plant disease classification using the PlantVillage dataset. A pretrained ResNet-18 is established as the initial supervised baseline, achieving **98.8%** test accuracy and a **97.9%** macro-F1 score on a three-class Potato disease subset after only three training epochs on CPU. The distilled student model (MobileNetV3-Small) and the full 38-class experiments are planned for the next phase. All code, configuration files, and experiment scripts are maintained in the project repository with full reproducibility.

2 Introduction

Automated detection of plant diseases from leaf images has the potential to assist small-holder farmers who lack access to agronomists, particularly if such systems can run on inexpensive mobile or edge hardware. Deep learning models can match or exceed expert-level accuracy on curated datasets [1], yet models like ResNet-50 or EfficientNet carry millions of parameters and impose latency costs that are impractical on devices with limited compute and battery budgets.

Knowledge distillation [2] addresses this gap by training a compact *student* network to mimic the output distribution of a larger, more accurate *teacher*. Rather than learning only from hard one-hot labels, the student also learns from the teacher’s softened probability vectors, which encode inter-class similarity and carry richer supervisory signal than a single correct-class indicator.

This project asks:

Can a distilled MobileNetV3-Small student for PlantVillage classification maintain competitive accuracy while improving parameter count, model size, and inference latency compared with its ResNet-50 teacher?

The present report covers the **progress checkpoint**, which establishes a reproducible data pipeline, a configurable model factory, and a supervised ResNet-18 baseline on a fast-running Potato-disease subset.

3 Related Work

3.1 Plant Disease Classification

Mohanty et al. [1] demonstrated that a CNN trained on the PlantVillage dataset could identify 26 diseases across 14 crop species with over 99% accuracy in controlled conditions. Subsequent work has explored data augmentation [6], semi-supervised learning, and transfer learning from ImageNet to improve generalisation to real field conditions.

3.2 Knowledge Distillation

Hinton et al. [2] introduced the concept of soft targets with temperature scaling to transfer the generalisation capacity of a large model to a smaller one. The distillation objective

blends a hard-label cross-entropy term with a Kullback–Leibler divergence between softened student and teacher distributions:

$$\mathcal{L} = \alpha \cdot T^2 \cdot \text{KL}\left(\sigma\left(\frac{z_s}{T}\right) \parallel \sigma\left(\frac{z_t}{T}\right)\right) + (1 - \alpha) \cdot \text{CE}(z_s, y) \quad (1)$$

where z_s and z_t are the student and teacher logits, T is the temperature, α is the soft-target weight, and y is the ground-truth label. The T^2 factor compensates for the reduced gradient magnitude caused by temperature scaling.

3.3 Efficient Architectures for Edge Deployment

MobileNets [3] use depth-wise separable convolutions to drastically reduce multiply-accumulate operations. MobileNetV3 [4] further incorporates hard-swish activations and squeeze-and-excitation modules found by neural architecture search, achieving a strong accuracy–latency trade-off on ImageNet. EfficientNet [5] applies compound scaling to balance depth, width, and resolution, offering another competitive teacher candidate.

4 Dataset and Preprocessing

4.1 PlantVillage

PlantVillage [1] is a publicly available dataset of healthy and diseased leaf images across 38 plant–disease combinations. Images are collected under controlled lighting against uniform backgrounds. The full dataset contains approximately 54,000 RGB images; individual class sizes vary considerably.

For the progress checkpoint, training is restricted to the three **Potato** classes to enable rapid iteration:

- Potato___Early_blight (133 test samples)
- Potato___Late_blight (102 test samples)
- Potato___healthy (23 test samples)

4.2 Train / Validation / Test Split

The dataset is not provided with pre-defined splits in the downloaded archive. We apply a random split with a fixed seed (42) to obtain:

Table 1: Dataset split sizes for the Potato subset (1,721 images total).

Split	Samples	Fraction
Train	1,205	70%
Validation	258	15%
Test	258	15%

4.3 Preprocessing and Augmentation

All images are resized to 224×224 pixels and normalised using ImageNet channel means and standard deviations ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

Training images additionally receive:

- Random horizontal flip (probability 0.5)
- Random rotation up to $\pm 10^\circ$

Validation and test images use only resize and normalise transforms. Labels are remapped to a contiguous $0 \dots C - 1$ range after subset filtering so that the cross-entropy loss always receives valid targets.

5 Methodology

5.1 Model Architecture

The model factory (`src/models/__init__.py`) supports four architectures loaded with pretrained ImageNet weights via the `torchvision` weights API. The final classification layer is replaced with a linear layer of size $d_{\text{in}} \times C$, where C is the number of target classes.

Table 2: Supported model architectures.

Role	Model	Approx. Parameters
Baseline / Teacher candidate	ResNet-18	11.2 M
Teacher	ResNet-50	25.6 M
Teacher candidate	EfficientNet-B0	5.3 M
Student	MobileNetV3-Small	2.5 M

5.2 Supervised Baseline Training

The baseline training procedure (`src/train_baseline.py`) follows a standard supervised protocol:

- **Loss:** Cross-entropy
- **Optimiser:** Adam, $\text{lr} = 10^{-3}$, weight decay $\lambda = 10^{-4}$
- **Epochs:** 3 (checkpoint scope)
- **Batch size:** 32

Device selection is automatic at runtime: $\text{CUDA} \succ \text{Apple MPS} \succ \text{CPU}$, with no code changes required between machines.

5.3 Distillation Loss (Next Phase)

Equation 1 is already implemented in `src/distillation.py`. Planned hyperparameters are $T = 4.0$ and $\alpha = 0.5$. Teacher and student will be trained on the full 38-class PlantVillage dataset in subsequent experiments.

6 Experiments and Results

6.1 Experiment Configuration

All experiments are driven by YAML configuration files under `configs/`. The baseline is fully reproducible by running:

```
python -m src.train_baseline --config configs/baseline_resnet18.yaml
```

Key hyperparameters for the current checkpoint are shown in Table 3.

Table 3: Hyperparameters for the ResNet-18 baseline experiment.

Hyperparameter	Value
Model	ResNet-18 (pretrained)
Image size	224×224
Batch size	32
Epochs	3
Learning rate	1×10^{-3}
Weight decay	1×10^{-4}
Optimiser	Adam
Seed	42

6.2 Training Dynamics

Table 4 shows per-epoch metrics on the training and validation splits. Loss converges rapidly; by epoch 3, validation accuracy and macro-F1 reach 100% on the validation split of the three-class Potato subset, indicating that pretrained ImageNet features transfer effectively to this domain.

Table 4: Training history – ResNet-18, Potato subset, 3 epochs.

Epoch	Train Loss	Val Loss	Val Acc	Val Macro-F1
1	0.1634	0.3046	0.919	0.944
2	0.1122	0.2408	0.911	0.904
3	0.0459	0.0043	1.000	1.000

6.3 Test Set Evaluation

Table 5 presents per-class precision, recall, and F1-score on the held-out test split (258 samples). The model misclassifies a single Early Blight sample and two Healthy samples, the latter likely due to the small Healthy class support (23 test samples).

Table 5: Per-class classification report on the test set (ResNet-18 baseline).

Class	Precision	Recall	F1-score	Support
Potato Early Blight	1.000	0.992	0.996	133
Potato Late Blight	0.971	1.000	0.986	102
Potato Healthy	1.000	0.913	0.955	23
Macro avg	0.990	0.969	0.979	258
Weighted avg	0.989	0.988	0.988	258

6.4 Summary Metrics

Table 6: Baseline experiment summary – ResNet-18, Potato subset.

Metric	Value
Test accuracy	98.84%
Test macro-F1	97.88%
Test weighted-F1	98.83%
Trainable parameters	11,178,051
Inference latency	20.83 ms (CPU, batch=1)
Training device	CPU
Epochs	3

6.5 Confusion Matrix

Figure 1 shows the confusion matrix on the test split.

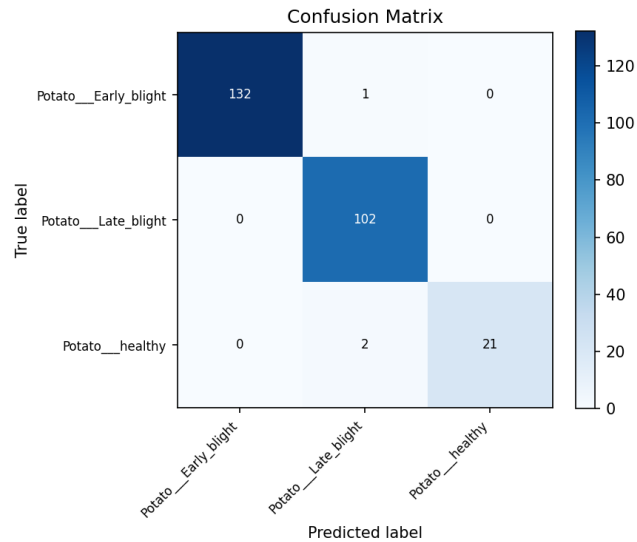


Figure 1: Confusion matrix for ResNet-18 baseline on the Potato test set (258 samples, 3 classes).

6.6 Planned Next-Phase Comparison

Table 7 will be completed in the next report once the MobileNetV3 supervised baseline, the ResNet-50 teacher, and the distilled student have been evaluated.

Table 7: Planned experiment comparison table (dashes = not yet run).

Model	Training	Acc	Macro-F1	Params	Latency (ms)
ResNet-18	Supervised	98.84%	97.88%	11.2M	20.83
MobileNetV3-Small	Supervised	—	—	2.5M	—
ResNet-50	Supervised	—	—	25.6M	—
MobileNetV3 (distilled)	Distillation	—	—	2.5M	—

7 Implementation Details

7.1 Repository Structure

```
knowledge-distillation-edge/
+-- configs/
|   +-- baseline_resnet18.yaml
|   +-- student_mobilenetv3.yaml
|   +-- distillation.yaml      (next-phase placeholder)
+-- data/README.md            (download instructions)
+-- outputs/
|   +-- resnet18_potato_subset/ (metrics, CSV, confusion matrix)
+-- reports/report.tex
+-- scripts/
|   +-- run_baseline.sh
|   +-- run_student.sh
+-- src/
|   +-- data/                  (dataset loading + label remapping)
|   +-- models/                (model factory)
|   +-- train_baseline.py      (training entry point)
|   +-- evaluate.py            (metrics + latency measurement)
|   +-- distillation.py        (distillation loss, next phase)
|   +-- utils/                 (config, seed, device helpers)
```

7.2 Reproducibility

- All experiments are seeded (Python, NumPy, PyTorch) via a single `seed` field in the YAML config (default: 42).
- No hard-coded file paths; `data_dir` and `output_dir` are config-driven.
- Automatic device selection runs identically on Apple Silicon (MPS) and NVIDIA CUDA without any code changes.
- Dataset download instructions are documented in `data/README.md`.

8 Conclusion and Future Work

This checkpoint establishes a clean, reproducible ML pipeline for plant disease classification on the PlantVillage dataset. The ResNet-18 supervised baseline achieves **98.84% test accuracy** and **97.88% macro-F1** on the Potato subset after three training epochs on CPU, confirming that ImageNet-pretrained features transfer effectively to this domain.

The immediate next steps are:

1. Run the MobileNetV3-Small supervised baseline and record its accuracy, parameter count, and inference latency.
2. Train a ResNet-50 teacher on the full 38-class PlantVillage dataset.
3. Train a MobileNetV3-Small student via knowledge distillation using the loss in Equation 1 with $T = 4.0$ and $\alpha = 0.5$.
4. Compare all models on accuracy, macro-F1, model size, parameter count, and single-image inference latency to quantify the compression–accuracy trade-off.
5. Explore post-training quantisation (INT8) as an additional edge optimisation strategy.

References

- [1] S. P. Mohanty, D. P. Hughes, and M. Salathe, “Using Deep Learning for Image-Based Plant Disease Detection,” *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
- [2] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” *NIPS Deep Learning Workshop*, 2015.
- [3] A. G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv:1704.04861*, 2017.
- [4] A. Howard et al., “Searching for MobileNetV3,” *IEEE/CVF ICCV*, pp. 1314–1324, 2019.
- [5] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” *ICML*, 2019.
- [6] Y. Toda and F. Okura, “How Convolutional Neural Networks Diagnose Plant Disease,” *Plant Phenomics*, 2019.